Pool-Based Entity Resolution for Mixed-Script CJK Records: Phased Matching with Data Completeness Separation

A Preprint

Augustin Chan
aug@iterative.day
Independent Researcher

October 17, 2026

Abstract

Entity resolution for CJK enterprise records faces two independent problems: mixed-script variation (Traditional Chinese, Simplified Chinese, Latin romanization) and data incompleteness. We present Dataline, an open-source Rust-native ER engine that combines independent phonetic, visual, and normalization signals through a deterministic rule engine. Its pool-based architecture separates phone-corroborated candidate groups from ambiguous district and name-only cohorts. Ten hash passes store group members without expanding all within-group pairs. On a synthetic benchmark of 1 million persons (2.7 million records), the current accepted-link graph achieves 99.9986% pair precision at 38.4% pair recall. Name-only passes contain 27.6 million implicit candidate pairs, of which only 0.026% refer to the same person. Holding these cohorts unresolved limits comparison work and unsupported merges, but early consumption and representative-based validation can also miss valid links. Grouped records are not necessarily resolved, and the pipeline provides no completeness guarantee. We retain historical results separately and distinguish accepted links from review candidates. The engine compiles to WebAssembly and is available under Apache 2.0.

Keywords: entity resolution, CJK matching, cross-script, multi-signal, pool-based, Hong Kong names, Cantonese romanization, open source

1 Introduction

Entity resolution (ER)—identifying records that refer to the same real-world entity—is a foundational problem in data integration [1]. For enterprises operating in Hong Kong, Taiwan, and mainland China, ER is complicated by the CJK script system: the same customer may appear as 陳大文 in a CRM system, `CHAN Tai Man` on an HKID card, 陈大文 in a mainland partner system, and `Peter Chan` in an English-only database.

A common production pattern for CJK MDM (Master Data Management) addresses the script problem by transliterating characters to Latin, then applying phonetic algorithms like NYSIIS [13]. This approach loses information at each stage: pinyin is many-to-one (multiple unrelated characters romanize identically), NYSIIS collapses Chinese consonant distinctions (zh/z/j, ch/c/q), tones are discarded entirely, and visual-similarity errors (OCR misreads, wrong radical) are invisible to phonetic matching. More fundamentally, such pipelines typically treat all records uniformly regardless of which corroborating fields are present—feeding sparse records (name only, or name+district) through the same pairwise engine as rich records, generating false positives at scale.

We present two contributions:

1. A *multi-signal* matching architecture for CJK names that computes phonetic, visual (stroke-based), and normalization (Simplified ↔ Traditional) signals independently, combining them after scoring via a deterministic rule engine. This catches error types that phonetic-only engines miss entirely, while providing traceable match decisions.

2. A *pool-based* architecture with two novel elements: using expected per-name collision count as a formal routing criterion before matching begins, and treating UNRESOLVED as a protected first-class pipeline state rather than a residual. Records whose corroborating fields have low expected collision count (phone, national ID) form validated anchor clusters. Records with only high-collision corroborators (name only, name+district) are classified as UNRESOLVED cohorts—never compared pairwise against each other, never merged at low confidence. The three output states (MERGED, UNRESOLVED, SINGLETON) reflect downstream business reality: an UNRESOLVED cohort is not a failure—it is a record awaiting enrichment, not a human reviewing 118 candidate pairs.

The engine is implemented in Rust for performance (1.7µs per multi-signal comparison), compiles to WebAssembly for browser demos, and stores all results in SQLite for transparent quality analysis.

2 Related Work

2.1 Entity Resolution at Scale

Christophides et al. [1] survey end-to-end ER for big data, identifying blocking, matching, and clustering as the three core stages. Papadakis et al. [8] comprehensively survey blocking and filtering techniques. Recent work on progressive ER [3] replaces global sorting with stochastic bipartite maximization for linear-time candidate prioritization. Auto-configuring pipelines [7] use Bayesian optimization to tune blocking and matching parameters without ground truth.

2.2 CJK Name Matching

Cross-script name matching has been studied primarily in the context of transliteration. An empirical study of Chinese name matching [5] achieved F-scores of 96% using syllable-level comparison. Syllable-based maximum matching [15] aligns English and Chinese names bidirectionally. The HK name linkage study [12] found that standardized romanization (Jyutping) improves linkage precision and recall compared to the non-standardized HK Government romanization. However, these approaches focus on transliteration accuracy rather than enterprise-scale ER with multiple data quality levels.

2.3 Incremental and Cluster-First ER

Simonini et al. [10] propose entity resolution on demand with query-driven matching. Fan et al. [2] formalize incremental ER on rules and data. Saeedi et al. [9] explore optimized cluster assignment for multi-source ER. The attractor (or *driver record*) pattern—where one record in a cluster acts as a reference point against which candidates are compared—is a well-established concept in commercial MDM. Our contribution is not the attractor mechanism itself, but the use of expected collision count to determine which records qualify as attractors at all, and the routing of high-collision records to UNRESOLVED cohorts rather than the matching pipeline.

2.4 Embedding-Based Blocking

Wang et al. [14] propose neural locality-sensitive hashing for learned blocking keys. Ma et al. [4] combine blocking with retrieval-augmented generation for LLM-based matching. Pre-trained embeddings for ER have been extensively evaluated [11]. Embedding-based blocking imposes a finite candidate horizon: only the top-k nearest neighbours are retained for downstream matching. Although larger k values may improve empirical recall, completeness remains non-deterministic and computational cost scales linearly with candidate set size. Dataline's deterministic grouping also provides no complete-recall guarantee: key selection, early consumption, and representative-based validation restrict which true links can be recovered.

3 Multi-Signal CJK Matching

3.1 Three Independent Signals

Rather than collapsing CJK characters to Latin for phonetic comparison, we compute three independent signals per character pair:

1. **Phonetic similarity**: character → Jyutping (Cantonese) or pinyin (Mandarin) → DimSim-style 2D coordinate distance [6]. Consonants and vowels are placed in a coordinate space where articulatorily similar sounds are close.

2. **Visual similarity**: stroke sequence comparison using a 20,901-character dictionary derived from FuzzyChinese. Characters sharing strokes (same radical, similar structure) score high via normalized Levenshtein distance on stroke sequences.
3. **Normalization**: Simplified ↔ Traditional character mapping using OpenCC dictionaries (8,093 entries). Binary signal: either they normalize to the same character or they don't.

These signals catch different error types (Table 1). A phonetic-only engine misses OCR/stroke errors entirely—the visual signal catches them.

Table 1: Error types and signal responses

Error type	Phonetic	Visual	Example
Phone dictation	HIGH	LOW	陳 chén → 程 chéng
OCR / stroke error	LOW	HIGH	陳 chén → 陣 zhèn
S ↔ T variant	MATCH	MATCH	陳 (trad) → 陈 (simp)

3.2 Cross-Script Matching via HK Government Romanization

For matching CJK characters against Latin romanizations, we use the Hong Kong Government Romanization standard —the actual system used on HKID cards. We integrate 11,612 character-to-romanization mappings generated from the `cantoroman` library, providing 22,521 romanization variants. This replaces ad-hoc Jyutping-to-capitalized approaches used in earlier systems.

3.3 Rule-Based Matching with Node DAG

Instead of continuous threshold-based scoring, match decisions are made by deterministic rules that combine precomputed component evaluations (nodes). Twelve node types are evaluated once and shared across ten rules via boolean logic:

- **Name-sufficient rules** (R1–R3): exact match, S ↔ T variant, synonym—no corroboration needed
- **Corroboration-required rules** (R4–R10): cross-script romanization, high Jaro-Winkler, partial given name —require phone/address/DOB agreement to upgrade confidence

Each decision is traceable to a named rule: "R3d: family match + given-name signal + phone match" rather than "score 0.73 > threshold 0.7."

4 Pool-Based Matching Architecture

4.1 Motivation: Incomplete Data Is the Norm

Enterprise data sources vary dramatically in completeness. CRM systems typically have partial phone coverage; billing systems tend toward higher completeness; legacy systems are often sparse; marketing lists frequently contain nothing but a name. Matching sparse records against each other produces massive false positives (name collision) and bogs down the pipeline.

4.2 Two-Pool Separation

Records are classified by the *expected collision count* of their corroborating fields. Formally, for a record with name n and field f taking value v, we define:

$$C(f, v, n) \;=\; \mathbb{E}\big[\,\big|\{r \in \mathcal{R} \mid r.\textit{name} = n \;\wedge\; r.f = v\}\big|\,\big]$$

where the expectation is over the target record population $\mathcal{R}$. In plain terms: $C(f, v, n)$ measures how many same-name records we expect to share corroborating value v for field f. Values near 1 indicate near-unique corroboration; values much larger than 1 indicate the field cannot safely distinguish individuals sharing that name. A field qualifies as a *low-collision corroborator* if $C(f, v, n) \lesssim \theta$ for some threshold θ close to 1. In practice, θ is calibrated to the maximum tolerable false merge rate for the deployment domain: financial compliance environments may set $\theta = 1$ (at most one expected collision per name group), while lower-stakes applications may permit $\theta = 2$ or 3. We use

collision count rather than Shannon entropy because it maps directly to false positive rate: if $C \approx 1$, a name+field match approaches uniqueness; if $C \gg 1$, the field cannot distinguish persons sharing that name and pairwise matching will be dominated by false positives.

- **Pool A (rich)**: records with at least one corroborating field whose expected collision count is close to one when combined with name. In practice: phone, email, national ID, or date of birth.[1]
- **Pool B (sparse)**: records whose only corroborating evidence has expected collision count $\gg 1$ (district alone, $\sim$6) or no corroboration at all (name only, $\sim$118 per name in the benchmark).

The collision-count formulation motivates the design. The current batch implementation uses fixed phone-backed versus district/name-only grouping keys; it does not estimate this expectation or calibrate a threshold per record.

Pool A records form candidate anchor groups through multi-field exact grouping; only phone-corroborated groups are validated in Phase 2a. The benchmark contains phone and district only, so phone is the sole low-collision corroborator used here. District and name-only groups remain UNRESOLVED cohorts and are not compared internally. These are design-level interpretations: the current SQLite cluster table stores candidate memberships without explicit MERGED/UNRESOLVED/SINGLETON status values. Accepted identity links are the output rows classified as `match`; rows classified as `possible` require review. Cohort membership alone must not be interpreted as identity.

4.3 Phased Processing Pipeline

1. **Phase 1: Multi-field exact grouping** (zero pairwise comparisons). The current ordered keys are: exact name+phone+district; exact name+phone; S↔T name+phone; stripped name+phone; romanised name+phone; exact name+district; S↔T name+district; romanised name+district; stripped name; romanised name. Each hash pass stores member indices rather than enumerating pairs. Grouped records skip later passes, except that later phone passes may add new members to existing phone groups; two existing groups are never merged.
2. **Phase 2a: Phone-corroborated validation**. Each phone-group member is compared against the richest representative, giving at most $k - 1$ comparisons for a group of size k. Only decisions at `High` or above are emitted as accepted matches. District and name-only cohorts are skipped.
3. **Phase 2b: Attractor assignment**. Only ungrouped records with phones are eligible, and only phone-corroborated groups can attract them. Phonetic surname shortlists targets; different phone numbers are rejected before rule evaluation. Grouped records, including rejected Phase 2a members and unresolved cohorts, are not retried.
4. **Stage 2: Pairwise remainder**. Ungrouped records are blocked by phonetic surname+district or surname+last four phone digits, falling back to surname alone when both fields are absent. Pairs are enumerated within each residual block, and blocks over 5,000 records are skipped. Phone suffixes retrieve candidates, not establish identity. The pipeline executes once rather than iterating to convergence.

5 Hong Kong Name Formats

Hong Kong names present unique parsing challenges. The same person appears as:

- 陳大文先生 (formal Chinese with honorific)
- `CHAN Tai Man` (HKID romanization, surname first in ALL CAPS)
- `CHAN Tai Man, Peter` (HKID with English alias after comma)
- `Peter Chan` (English order, surname last)
- 陈大文 (Simplified Chinese, mainland system)
- 阿文 (informal Cantonese with prefix)

We implement a component-level name parser that handles: compound surnames (歐陽, Au-Yeung), CJK honorific/prefix stripping (先生, 阿/小/老), HKID format detection (ALL CAPS surname first), comma-separated English names, and hyphenated given names. An 80-entry HK surname dictionary enables surname-first vs surname-last disambiguation. Correct component extraction is a prerequisite for blocking key quality: a parser that conflates 陳 (surname) with 大文 (given name) will fail to produce matching keys across systems, causing missed links that no downstream matching signal can recover.

[1] For an 8-digit HK mobile number: $\sim$0.001 expected collisions per name group. For date of birth (29,200 possible values over an 80-year span): $\sim$0.004. Both satisfy the threshold. District (19 HK districts, $\sim$6 persons per name+district group): $\sim$6 expected collisions—insufficient as a sole corroborator, though useful for Phase 1 blocking.

6 Experimental Evaluation

6.1 Synthetic Benchmark

We generate a dataset of 1 million persons (2,691,721 records) across four source systems: CRM (Chinese + honorific, 80% phone), Billing (HKID romanization, 100% phone), Legacy (Simplified Chinese, 40% phone, 50% inclusion rate), English (English name only, 70% phone, 19% inclusion rate). Given names are drawn as atomic bigram pairs from gender-stratified pools (87 male classic, 70 female classic, 68 Gen Y/Z across genders) with HK Government Romanization for cross-script consistency. Surnames follow a Zipf distribution over 50 HK surnames plus compound surnames (歐陽, 司徒).

6.2 Historical Results

Table 2: Historical pipeline results before the October 2026 recall fixes (2,691,721 records)

Phase	Comparisons	Matches	TPs	Precision	Recall
Phase 1 (hash)	0	—	—	—	—
Phase 2a (phone-corroborated)	21,176	20,878	20,877	100.0%	0.8%
Phase 2b (attractor)	1,945,780	14,347	13,636	95.0%	0.6%
Stage 2 (pairwise remainder)	389,748	112,299	1	0.0%	—
Total (2a+2b)	—	**35,225**	**34,513**	**98.0%**	**1.4%**

The historical Phase 2a result contains one false positive; 100.0% is rounded precision, not an error-free guarantee. Historical Phase 2b achieved 95.0% direct-edge precision, and the combined Phase 2a+2b precision was 98.0%. The Stage 2 row illustrates the ambiguity of the residual population under that code version. It is not a controlled ablation proving how every alternative matching system would behave.

6.3 Current Results and Metric Scope

A fresh run on 2026-10-16 reproduced the current README counts on the unchanged 1M-person input: 825,411 accepted direct edges in Phase 2a, including six false positives; zero Phase 2b comparisons or assignments; and two Stage 2 review candidates, both incorrect, after 264 comparisons. The six accepted false positives have the same full name and phone but different ground-truth identities. Accepted direct-edge precision is 99.999273%.

The accepted-link graph contains 952,357 connected record pairs, of which 952,344 are correct and 13 are wrong: 99.9986% pair precision and 38.4% pair recall against 2,479,642 true pairs. The default `scripts/pair_metrics.py` includes review candidates for compatibility; that graph contains 952,360 pairs, 16 wrong, and 99.9983% pair precision at the same recall. Use `--classification match` for accepted links only. Direct-edge counts and transitive pair counts are different metrics; false edges can amplify through connected components.

Phase 1 consumed 99.9% of records, but consumption means grouping rather than resolution. Phone-corroborated candidate groups contain 1,114,564 true pairs (44.9% candidate recall), while accepted links recover 38.4%. The name-only stripped pass stores 565,704 records representing 27,414,408 possible pairs, only 7,067 of which are true; the final romanised pass stores 34,726 records representing 148,880 possible pairs, only one true. Combined candidate purity is 0.026%. These counts use $k(k-1)/2$ per group and ground-truth person counts, without enumerating pairs. They measure candidate ambiguity, not the precision an all-pairs rule-engine run would achieve. They also exclude true pairs crossing group boundaries. The accompanying `scripts/cohort_metrics.py` reproduces them from a fresh database and its pass log.

On the shared-phone variant, the README reports 34.9% pair recall and 99.998% pair precision using the default graph definition. A number appearing with more than two distinct surname sounds, or on more than eight records, is treated as absent phone evidence. These are synthetic benchmarks, not a validation on enterprise records.

6.4 Historical Confidence Strategies

The historical evaluation compared a strict phone-corroborated subset with a broader auto-merge tier. These historical strategy counts must not be read as the current batch CLI's configuration or as a guarantee that false merges cannot occur. Current output classifications and accepted-link metrics are reported separately above.

6.5 Historical Performance

The original performance report processed 2,691,721 records on a 20-core commodity machine:

- Phase 1 (hash grouping): 17 seconds, 98.1% records consumed
- Phase 2a (phone-corroborated validation): $<$1 second (21,176 comparisons)
- Phase 2b (attractor assignment): 12 seconds (parallel, 20 threads)
- Stage 2 (pairwise remainder): 5 seconds (389,748 pairs)
- Total: 66 seconds (1.1 minutes)

These historical timings are not current performance guarantees. The 2026-10-16 verification took 102 seconds with 12 Rayon threads in the available Linux environment; the README records a separate roughly 68-second run on an Apple M2 Max. Hardware, grouping policy, and output costs affect runtime.

Historical raw matching speed: 1.7µs per multi-signal CJK comparison (588K comparisons/sec single core). Phase 1 hash grouping consumes 98.1% of records, leaving only 52,275 for Phase 2b attractor assignment and 37,928 for Stage 2 pairwise—dramatically reducing comparison work relative to a flat pairwise approach.

7 Discussion

7.1 Interpreting Synthetic Benchmark Results

The synthetic name pool creates frequent collisions, making name equality insufficient evidence of identity. Historical Stage 2 results demonstrate this risk for that code version. The current name-only cohort counts measure ambiguity directly: 27,563,288 implicit pairs contain only 7,068 true pairs. Holding those cohorts unresolved avoids enumerating their quadratic pair sets. It does not eliminate all sparse-record comparisons: records outside every group still enter the residual pairwise stage.

The current system favours precision and misses 61.6% of true pair links. Missing phones and English-only names explain much of the gap, but not all of it: 57.7% of true input pairs have phones on both records before shared-phone filtering, 44.9% occur within phone-backed candidate groups, and 38.4% are recovered. The former 1.3% pair recall rose to 38.4% after correcting pass order, group extension, romanization keys, and dictionary gaps. Low recall therefore cannot be attributed solely to data completeness. Early consumption, grouping-key coverage, rule rejection, and representative-based comparison also restrict recall. These measurements do not isolate the causal recall cost of each restriction; per-pass candidate-recall analysis and controlled ablations remain needed. Grouped records can be reconsidered by a fresh run after enrichment, but the current batch pipeline does not implement an event-driven resolution service. Validation on real enterprise data remains future work.

7.2 Why Pool Separation Matters

Enumerating every pair within a cohort of k members requires $O(k^2)$ work; storing the cohort requires $O(k)$ member indices. For a fixed number of hash passes, grouping is expected linear work in record count and key lengths. This does not make the entire pipeline linear: Phase 2b searches candidate attractors, and residual blocks enumerate pairs. Skipping residual blocks above 5,000 members also sacrifices potential links. Integrity and coverage must be measured together rather than inferred from grouping speed.

7.3 Comparison with Commercial Engines

Commercial MDM engines use a transliterate → NYSIIS → compare pipeline that loses information at each stage. Our multi-signal approach preserves the original characters, computing phonetic and visual signals independently. The rule-based decision engine (vs. continuous threshold scoring) provides auditable match decisions: customers see "R3d: family match + given-name signal + phone match" rather than an opaque score.

The pool-based architecture addresses a gap in common approaches: many commercial MDM platforms converged toward a single-golden-record model where multiple views require explicit configuration. Our computed-view model —where the golden record is derived on-demand from linked records via survivorship rules—makes multiple views a natural consequence of the architecture rather than a separate concern.

8 Conclusion

The primary contribution is architectural: preserving ambiguous cohorts alongside corroborated identity links. On the current synthetic benchmark the accepted-link graph achieves 99.9986% pair precision at 38.4% pair recall. Ten hash passes avoid expanding name-only cohorts into 27.6 million pairs, but early consumption and representative-based validation also restrict coverage. Accepted matches, review candidates, and candidate memberships must remain distinct when interpreting outputs. The multi-signal engine provides auditable cross-script matching, compiles to WebAssembly, and is available under Apache 2.0.

Future work: ordered cross-script syllable alignment (comparing Jyutping/pinyin sequences directly against romanized tokens to improve recall for records without phone corroboration), and validation on real enterprise data where phone coverage and name pool sizes differ from the synthetic benchmark.

Broader vision —**Chronicle MDM.** Dataline is the matching engine layer of a larger process model. In production, records are not a static dataset but *living documents*: they accumulate fields over time, carry resolution state (MERGED, UNRESOLVED, SINGLETON), and are progressively resolved by business events rather than batch runs. Under this model, UNRESOLVED records sit passively in name cohorts—not actively managed, not merged—waiting for a triggering event (a phone number collected during KYC, a biometric confirmation, a transaction linking to a known golden record) that provides sufficient corroboration to route them. A golden record built from high-confidence matches becomes a *transitive attractor*: its accumulated fields extend the matching reach to sparse records that were unresolvable in isolation. Each pipeline run produces richer attractors, improving resolution in subsequent runs. This incremental, event-driven architecture—where the engine presented here provides the corroboration layer—is the basis of Chronicle MDM, a planned open-source platform for living customer identity management.

Availability: Source code, benchmarks, and live demo at `https://github.com/digital-rain-tech/dataline`

References

[1] Vassilis Christophides, Vassilios Efthymiou, Themis Palpanas, George Papadakis, and Konstantinos Stefanidis. An overview of end-to-end entity resolution for big data. *ACM Computing Surveys*, 53(6):1–42, 2020.

[2] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, and Wenyuan Yu. Incremental entity resolution on rules and data. *The VLDB Journal*, 23:135–160, 2013.

[3] Dimitrios Karapiperis, George Papadakis, Themis Palpanas, and Vassilios S. Verykios. Sper: Accelerating progressive entity resolution via stochastic bipartite maximization. *arXiv preprint*, 2025.

[4] Chuangtao Ma, Zeyu Zhang, and Arijit Khan. Cost-efficient rag for entity matching with llms: A blocking-based exploration. *arXiv preprint*, 2025.

[5] Inderjeet Mani, Yifan Hu, Jianping Dong, and Laney Zhao. An empirical study of chinese name matching and applications. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics*, pages 377–383. Association for Computational Linguistics, 2015.

[6] Jiawei Min and Yiming Zhao. Dimsim: A chinese phonetic similarity algorithm. `https://github.com/Wikipedia2008/DimSim`, 2018.

[7] Konstantinos Nikoletos, Vassilios Efthymiou, George Papadakis, and Konstantinos Stefanidis. Auto-configuring entity resolution pipelines. *IEEE Access*, 2025. Also available as arXiv:2503.13226.

[8] George Papadakis, Ekaterini Ioannou, Claudia Thanos, and Themis Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys*, 53(2):1–42, 2020.

[9] Alieh Saeedi, Eric Peukert, and Erhard Rahm. Incremental multi-source entity resolution for knowledge graph completion. In *Extended Semantic Web Conference (ESWC)*, pages 393–408. Springer, 2020.

[10] Giovanni Simonini, Luca Gagliardelli, Sonia Bergamaschi, and H. V. Jagadish. Entity resolution on-demand. *Proceedings of the VLDB Endowment*, 15(7):1506–1518, 2022.

[11] Dimitrios Skoutas et al. Pre-trained embeddings for entity resolution: An experimental analysis. *Proceedings of the VLDB Endowment*, 16(9):2225–2238, 2023.

[12] Tymoteusz Strojny and Maciej Berešewicz. Data note: Alternative name encodings—using jyutping or pinyin as tonal representations of chinese names for data linkage. *International Journal of Population Data Science*, 2025.

[13] Robert L. Taft. Name search techniques. Technical report, New York State Identification and Intelligence System, 1970. Albany, NY. Describes the NYSIIS algorithm.

[14] Runhui Wang, Luyang Kong, and Yefan Tao. Neural locality sensitive hashing for entity blocking. In *Proceedings of NAACL-HLT*, 2024.

[15] Ching-Long Yeh and Tung-Cheng Wu. English-chinese name transliteration with bi-directional syllable-based maximum matching. In *Proceedings of the 23rd Conference on Computational Linguistics and Speech Processing*, 2011.